

MHTs Root Explanation

Absolutely. This problem looks complicated because of the wording, but the actual question is quite simple.

1. What is the question asking?

You are given an **undirected tree**.

You are allowed to pick **any node as the root**.

After choosing a root, the tree gets a height:

Height = the maximum number of edges from the root to any leaf.

Your job is:

Try every possible node as the root and find which node(s) produce the smallest height.

Those nodes are called **Minimum Height Tree roots (MHTs)**.

2. First understand with a diagram

Take Example 1:

```
edges = [[1,0], [1,2], [1,3]]
```

The tree looks like:



Now we can choose **any node** as root.

Case 1: Root = 1



Distances from root 1:

```

1 → 0 = 1
1 → 2 = 1
1 → 3 = 1

```

So:

```
height = 1
```

Case 2: Root = 0

Now imagine the same tree rooted at 0:



Distances:

```

0 → 1 = 1
0 → 2 = 2
0 → 3 = 2

```

Therefore:

```
height = 2
```

Case 3: Root = 2



```

  / \
0   3

```

Longest path:

```
2 → 1 → 0
```

or

```
2 → 1 → 3
```

So:

```
height = 2
```

Similarly, root 3 gives height 2.

Therefore:

root	height
0	2
1	1 ← minimum
2	2
3	2

So answer:

```
[1]
```

3. Example 2

```

n = 6
edges = [
    [3,0],
    [3,1],
    [3,2],
    [3,4],
    [5,4]
]

```

The tree is:

```

  0
  |
  3
 / | \
1  2  4
    |
    5

```

Let's calculate.

Root = 3

```

  3
 / | \
0  1  2
    |
    4
    |
    5

```

Actually structurally:

```

  3
 / | \
0  1  2
    |
    4
    |
    5

```

The important distances from 3 are:

```

3 → 0 = 1
3 → 1 = 1
3 → 2 = 1
3 → 4 = 1
3 → 5 = 2

```

Therefore:

height = 2

Root = 4



Distances:

```

4 → 3 → 0 = 2
4 → 3 → 1 = 2
4 → 3 → 2 = 2
4 → 5 = 1
  
```

So:

height = 2

Therefore both 3 and 4 give minimum height.

answer = [3,4]

4. Brute Force idea

The simplest possible approach is:

Try every node as the root.

For each possible root:

1. Make that node the root.
2. Run BFS/DFS from that node.
3. Find the farthest node.
4. The distance to that farthest node = height.
5. Keep track of the minimum height.
6. Store every root having that minimum height.

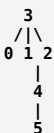
So conceptually:

```

Root 0 → calculate height
Root 1 → calculate height
Root 2 → calculate height
Root 3 → calculate height
...
Root n-1 → calculate height
      ↓
    Find minimum height
      ↓
    Return all roots having it
  
```

5. Why BFS/DFS gives us the height

Suppose:



Start BFS from 3.

distance[3] = 0



These are distance 1.

Then:

```
3
|
1
|
4
|
5
```

5 is distance 2.

So maximum distance from 3 is:

```
2
```

Therefore height when rooted at 3 is 2.

6. Brute Force Code

```
C++
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        // Special case
        if (n == 1) {
            return {0};
        }

        // Build adjacency list
        vector<vector<int>> graph(n);

        for (auto &edge : edges) {
            int u = edge[0];
            int v = edge[1];

            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        int minHeight = INT_MAX;
        vector<int> answer;

        // Try every node as root
        for (int root = 0; root < n; root++) {
            // Distance from current root
            vector<int> dist(n, -1);

            queue<int> q;

            // Start BFS from root
            q.push(root);
            dist[root] = 0;

            // BFS
            while (!q.empty()) {
                int node = q.front();
                q.pop();

                for (int neighbor : graph[node]) {
                    // Not visited yet
                    if (dist[neighbor] == -1) {
                        dist[neighbor] = dist[node] + 1;
                        q.push(neighbor);
                    }
                }
            }

            // Find height of tree rooted at 'root'
            int height = 0;

            for (int i = 0; i < n; i++) {
                height = max(height, dist[i]);
            }

            // Found a smaller height
            if (height < minHeight) {
                minHeight = height;
                answer.clear();
                answer.push_back(root);
            }

            // Another root with same minimum height
            else if (height == minHeight) {
                answer.push_back(root);
            }
        }
    }
};
```

```
    }  
    return answer;  
};
```

7. Let's understand the code step-by-step

Step 1: Build the graph

For:

```
edges = [[1,0], [1,2], [1,3]]
```

we create:

```
C++  
vector<vector<int>> graph(n);
```

Then:

```
C++  
graph[1] = {0, 2, 3}  
graph[0] = {1}  
graph[2] = {1}  
graph[3] = {1}
```

Because the tree is **undirected**, we add both directions:

```
C++  
graph[u].push_back(v);  
graph[v].push_back(u);
```

Step 2: Try every node

```
C++  
for (int root = 0; root < n; root++)
```

This means:

```
root = 0  
root = 1  
root = 2  
root = 3  
...
```

Every node gets a chance to become the root.

Step 3: BFS from that root

For example:

```
C++  
root = 1
```

We start:

```
C++  
dist[1] = 0;  
q.push(1);
```

Then BFS calculates:

```
dist[1] = 0  
dist[0] = 1  
dist[2] = 1  
dist[3] = 1
```

Therefore:

```
C++  
height = 1;
```

Step 4: Find the height

```
C++
```

```
int height = 0;
for (int i = 0; i < n; i++) {
    height = max(height, dist[i]);
}
```

Why maximum?

Because height is:

The longest distance from root to any node/leaf.

For root 1:

```
dist:
0 → 1
1 → 0
2 → 1
3 → 1
```

Maximum:

```
1
```

Therefore:

```
height = 1
```

8. How we maintain the answer

Initially:

```
C++
int minHeight = INT_MAX;
```

Suppose we get:

```
root = 0
height = 2
```

Then:

```
C++
minHeight = 2
answer = [0]
```

Next:

```
root = 1
height = 1
```

Since $1 < 2$:

```
C++
minHeight = 1
answer = [1]
```

Next:

```
root = 2
height = 2
```

Since:

```
2 > 1
```

ignore it.

So final:

```
[1]
```

9. Why this is called brute force

Because we don't use any special property of MHTs.

We simply say:

"Let me try every possible root."

For every root:

```
Run BFS
  ↓
Calculate height
  ↓
Compare with minimum
```

Very straightforward.

10. Time Complexity

There are n possible roots.

For **each root**, BFS visits every node and edge:

$O(n)$

So:

$n \text{ roots} \times O(n)$

Therefore:

$O(n^2)$

More precisely, BFS is $O(V + E)$.

Since this is a tree:

$E = n - 1$

so:

$O(V + E) = O(n)$

And we do it n times:

$O(n^2)$

Space:

$O(n)$

for the adjacency list, queue, and distance array.

The core intuition to remember

Don't initially think about the fancy optimized solution.

For the **brute force**, remember just this:

```
Every node can be the root
  ↓
Try each node
  ↓
Run BFS from it
  ↓
Maximum distance = height
  ↓
Find minimum height
  ↓
Return all nodes having that height
```

And the most important conceptual point:

MHT does NOT mean a tree with minimum number of nodes or minimum number of edges. It means the root is chosen such that the farthest leaf is as close as possible to that root.

Later, the optimized solution uses the "**peel leaves layer by layer**" idea and reduces this from $O(n^2)$ to $O(n)$.

Yes. The brute-force idea stays **exactly the same**. The only difference is:

- Before: for every root → **BFS**

- Now: for every root → DFS

The DFS will calculate the **maximum distance from the chosen root**.

DFS Brute Force Code

```
C++
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // DFS calculates the maximum distance
    // from 'node' to any node in its subtree/path
    int dfs(int node, int parent, vector<vector<int>>& graph) {
        int height = 0;
        for (int neighbor : graph[node]) {
            // Don't go back to the parent
            if (neighbor == parent)
                continue;

            // Distance from current node to this neighbor
            int childHeight = dfs(neighbor, node, graph);

            // Take the longest path
            height = max(height, 1 + childHeight);
        }
        return height;
    }

    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        // Special case
        if (n == 1)
            return {0};

        // Build adjacency list
        vector<vector<int>> graph(n);

        for (auto& edge : edges) {
            int u = edge[0];
            int v = edge[1];

            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        int minHeight = INT_MAX;
        vector<int> answer;

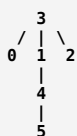
        // Try every node as root
        for (int root = 0; root < n; root++) {
            // Calculate height when 'root' is the root
            int height = dfs(root, -1, graph);

            // Found a smaller height
            if (height < minHeight) {
                minHeight = height;
                answer.clear();
                answer.push_back(root);
            }

            // Same minimum height
            else if (height == minHeight) {
                answer.push_back(root);
            }
        }
        return answer;
    }
};
```

How the DFS works

Suppose:

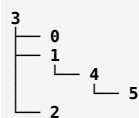


Take 3 as root.

We call:

```
C++
dfs(3, -1)
```

Then DFS explores:



For node 5:

```
dfs(5, 4) = 0
```

because 5 has no other child.

Then node 4:

```
dfs(4, 3)
child 5 gives:
1 + dfs(5, 4)
= 1 + 0
= 1
```

So:

```
dfs(4, 3) = 1
```

Then node 1:

```
dfs(1, 3)
1 + dfs(4, 1)
= 1 + 1
= 2
```

So:

```
dfs(1, 3) = 2
```

Finally node 3:

```
dfs(3, -1)
```

Its branches have heights:

```
0 → 0
1 → 2
2 → 0
```

Therefore:

```
height = max(1 + 0,
             1 + 2,
             1 + 0)
= 3
```

So when root is 3, height is actually **3** for this tree.

The key formula

The important line is:

```
C++
height = max(height, 1 + childHeight);
```

Why +1?

Because moving:

```
current node → child
```

adds **one edge**.

So if the child has height 2:

```
current
|
child
|
...
|
leaf
```

the current node's height becomes:

```
1 + 2 = 3
```

One important difference from BFS

With BFS, we explicitly store distances:

```
C++
dist[neighbor] = dist[node] + 1;
```

With DFS, we don't need a distance array. The **return value of DFS itself represents the height**:

```
dfs(node)
  ↓
maximum distance from node to a leaf
```

So the overall brute-force complexity is still:

```
O(n²) time
O(n) space
```

because we're still running a complete DFS from **every possible root**.

Yes. The optimal solution is much more interesting because we **don't try every node as root**.

The key idea is:

The best root(s) must be in the center of the tree.

And we find that center by **repeatedly removing the leaf nodes**, which is essentially applying **topological-sort/Kahn's algorithm on an undirected tree**.

1. First understand the intuition

Consider this tree:

```

  0
  |
  3
 / | \
1  2  4
   |
   5
```

The longest path is:

```
1 → 3 → 4 → 5
```

The center of this path is:

```

1 → 3 → 4 → 5
  ↑   ↑
center
```

There are **two center nodes**:

```
3 and 4
```

And those are exactly the MHT roots:

```
[3, 4]
```

Why?

Because if you put the root at an **end** of the tree, some nodes will be very far away.

If you put the root closer to the **center**, the maximum distance to all leaves becomes smaller.

So:

MHT roots = center node(s) of the tree.

A tree can have:

- **1 center**
- or **2 centers**

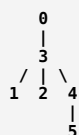
Never more than 2.

2. How do we find the center?

Here's the clever part.

Instead of calculating the height from every node, imagine that we are **peeling the tree from outside toward the center**.

Look at this tree:



The nodes having only **one connection** are called **leaves**.

Here:

leaves = 0, 1, 2, 5

These are the outermost nodes.

Remove them:



Now:

3 and 4

are the remaining nodes.

These are the centers.

Therefore:

answer = [3, 4]

3. Why can we safely remove leaves?

This is the most important intuition.

Suppose you have:

A --- B --- C --- D --- E

If we choose A as root:

A → B → C → D → E

height = 4

Clearly A is not a good root.

What about B?



Height:

B → A = 1
B → C → D → E = 3

```
height = 3
```

What about c?



Height:

```

C → B → A = 2
C → D → E = 2

height = 2

```

So the center is better.

Therefore, nodes far away from the center can never be the optimal root.

The leaf-removal process simply removes these useless outer layers.

4. Where does Topological Sort come in?

This is where the terminology can be confusing.

Normally, **topological sort** is used on a **directed acyclic graph (DAG)**.

But here we have an **undirected tree**.

We use the **same idea as Kahn's algorithm**:

Process nodes whose degree becomes zero/one, remove them, and update their neighbors.

In a tree, the equivalent of an "indegree 0" node is basically a **leaf with degree 1**.

So we maintain:

```

C++
degree[node]

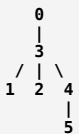
```

which tells us:

How many neighbors does this node currently have?

5. Initial state

For:



degrees are:

```

0 → 1
1 → 1
2 → 1
3 → 4
4 → 2
5 → 1

```

Therefore leaves are:

```
0, 1, 2, 5
```

Put them into a queue:

```
queue = [0, 1, 2, 5]
```

This is similar to Kahn's algorithm:

```

Find nodes with no remaining dependency
↓
Put them into queue
↓

```

```
Process them
↓
Update neighbors
↓
New nodes become eligible
```

Here "eligible" means:

```
degree == 1
```

6. Remove the first layer

Take:

```
0
```

Remove it.

Its neighbor is:

```
3
```

So:

```
degree[3] --
```

Originally:

```
degree[3] = 4
```

Now:

```
degree[3] = 3
```

It is still not a leaf.

Take:

```
1
```

Remove it.

Again neighbor:

```
3
```

Now:

```
degree[3] = 2
```

Take:

```
2
```

Remove it.

Now:

```
degree[3] = 1
```

So 3 has now become a leaf.

We add:

```
3
```

to the queue.

Take:

```
5
```

Its neighbor is 4.

So:

```
degree[4] = 1
```

Now 4 also becomes a leaf.

So:

```
queue = [3, 4]
```

7. Why do we stop here?

Now only:

```
3 --- 4
```

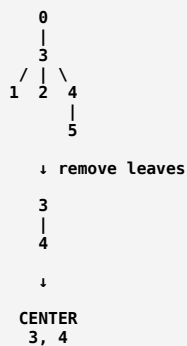
is left.

These are the center nodes.

If we remove them too, we would lose the answer.

So the algorithm works **layer by layer**.

Original tree



8. The most important trick: remainingNodes

We maintain:

```
C++
remainingNodes = n;
```

Every time we remove a layer:

```
C++
remainingNodes -= numberOfLeaves;
```

For our example:

Initially:

```
remainingNodes = 6
```

First layer:

```
leaves = 4
```

After removing:

```
remainingNodes = 6 - 4 = 2
```

Now only 2 nodes remain.

Therefore:

```
answer = [3, 4]
```

We stop because:

```
remainingNodes <= 2
```

Why 2?

Because a tree can have at most **two center nodes**.

9. Why only 1 or 2 nodes remain?

Think about a path:

A --- B --- C --- D --- E

Remove leaves:

A E
↓ ↓
B --- C --- D

Again:

B D
↓ ↓
C

One node remains:

C

So:

one center

For an even-length path:

A --- B --- C --- D

Remove leaves:

B --- C

Two nodes remain:

B, C

So:

two centers

Therefore:

MHTs = final 1 or 2 nodes

10. The complete intuition

Think of the tree as an onion 🍷.

The outermost layer contains leaves.

leaves
↓ ↓ ↓ ↓ ↓ ↓
inner nodes
↓
center

We repeatedly peel the outer layer:

Tree
↓
Remove leaves
↓
Smaller tree
↓
Remove leaves
↓
Smaller tree
↓
...
↓
1 or 2 nodes

Those final nodes are the **center(s)**.

And:

The center(s) of a tree are exactly the roots that produce Minimum Height Trees.

11. Why this is related to Topological Sort

The structure is almost exactly **Kahn's algorithm**.

Normal Kahn's algorithm

For a DAG:

```

indegree = 0
↓
put in queue
↓
remove node
↓
decrease indegree of neighbors
↓
new indegree 0 nodes enter queue

```

Here

For an undirected tree:

```

degree = 1
↓
put leaf in queue
↓
remove leaf
↓
decrease degree of neighbor
↓
new degree 1 nodes enter queue

```

So you can remember:

MHT optimal approach = Kahn's topological-sort style leaf peeling.

It's not literally a topological sort of the undirected tree, because topological ordering is formally defined for directed graphs. It's the **same queue + degree-reduction pattern** applied to leaves.

12. Optimal approach

Step 1

Build adjacency list.

```

C++
vector<vector<int>> graph(n);

```

Step 2

Calculate degree of every node.

```

C++
degree[i] = graph[i].size();

```

Step 3

Put all leaves into queue.

```

C++
if (degree[i] == 1)
    q.push(i);

```

Step 4

Remove leaves **layer by layer**.

For every leaf:

```

remove leaf
↓
decrease degree of its neighbor

```



```

    ↓
if neighbor becomes leaf
    ↓
put neighbor into queue

```

Step 5

Stop when:

```

C++
remainingNodes <= 2

```

Step 6

The nodes left in the queue are the MHT roots.

13. Complexity

This is where the big improvement happens.

Every node is removed **once**.

Every edge is processed only a constant number of times.

Therefore:

```

Time = O(V + E)

```

Since a tree has:

```

E = V - 1

```

we get:

```

Time = O(n)

```

Space:

```

O(n)

```

So:

```

Brute Force DFS:  O(n²)
Optimal:          O(n)

```

The key sentence to remember for interviews/DSA is:

Instead of trying every node as root, repeatedly remove all current leaf nodes. This peels the tree from the outside toward the center, and the final one or two nodes are the only possible minimum-height roots.

Sure. This is the **optimal O(n) leaf-peeling / Kahn's algorithm approach**.

```

C++
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        // If there is only one node,
        // that node itself is the center/root.
        if (n == 1) {
            return {0};
        }

        // Build adjacency list
        vector<vector<int>> graph(n);

        // degree[i] = number of current neighbors of node i
        vector<int> degree(n, 0);

        for (auto &edge : edges) {
            int u = edge[0];
            int v = edge[1];

            graph[u].push_back(v);
            graph[v].push_back(u);

            degree[u]++;
            degree[v]++;
        }

        // Queue contains all current leaf nodes
    }
};

```

```

queue<int> q;

for (int i = 0; i < n; i++) {
    // A leaf has only one neighbor
    if (degree[i] == 1) {
        q.push(i);
    }
}

// Number of nodes that are still remaining
int remainingNodes = n;

// Remove leaves layer by layer
while (remainingNodes > 2) {

    // Number of leaves in the current layer
    int leaves = q.size();

    // Remove this entire layer
    remainingNodes -= leaves;

    for (int i = 0; i < leaves; i++) {
        int leaf = q.front();
        q.pop();

        // Remove this leaf from the tree.
        // Its neighbors lose one connection.
        for (int neighbor : graph[leaf]) {
            degree[neighbor]--;

            // If neighbor now has only one
            // remaining connection, it becomes a leaf.
            if (degree[neighbor] == 1) {
                q.push(neighbor);
            }
        }
    }

    // The remaining 1 or 2 nodes are the centers
    // and therefore the MHT roots.
    vector<int> answer;

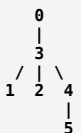
    while (!q.empty()) {
        answer.push_back(q.front());
        q.pop();
    }

    return answer;
}
};

```

The code flow to remember

For this tree:



Initially:

```

degree:
0 → 1 ← leaf
1 → 1 ← leaf
2 → 1 ← leaf
3 → 4
4 → 2
5 → 1 ← leaf

```

So:

```
queue = [0, 1, 2, 5]
```

Layer 1

```

C++
remainingNodes -= leaves;

```

```
6 - 4 = 2
```

After removing:

```
0, 1, 2, 5
```

we are left with:

```
3 --- 4
```

Since:

```
C++
remainingNodes <= 2
```

we stop.

The queue contains:

```
[3, 4]
```

Therefore:

```
answer = [3, 4]
```

The most important part of the code

This is the actual **leaf peeling**:

```
C++
for (int neighbor : graph[leaf]) {
    degree[neighbor]--;
    if (degree[neighbor] == 1) {
        q.push(neighbor);
    }
}
```

Meaning:

```
Remove leaf
↓
Its neighbor loses one edge
↓
If neighbor now has only 1 edge
↓
It becomes a new leaf
↓
Put it into queue
```

And this:

```
C++
while (remainingNodes > 2)
```

means:

Keep peeling outer layers until only the center node(s) remain.

So the complete pattern is:

```
Build graph
↓
Find all leaves
↓
Put leaves in queue
↓
Remove one complete layer
↓
Update degrees
↓
New leaves enter queue
↓
Repeat
↓
1 or 2 nodes remain
↓
Those are MHT roots
```

Complexity: $O(n)$ time and $O(n)$ space.

ChatGPT can make mistakes. Check important info.